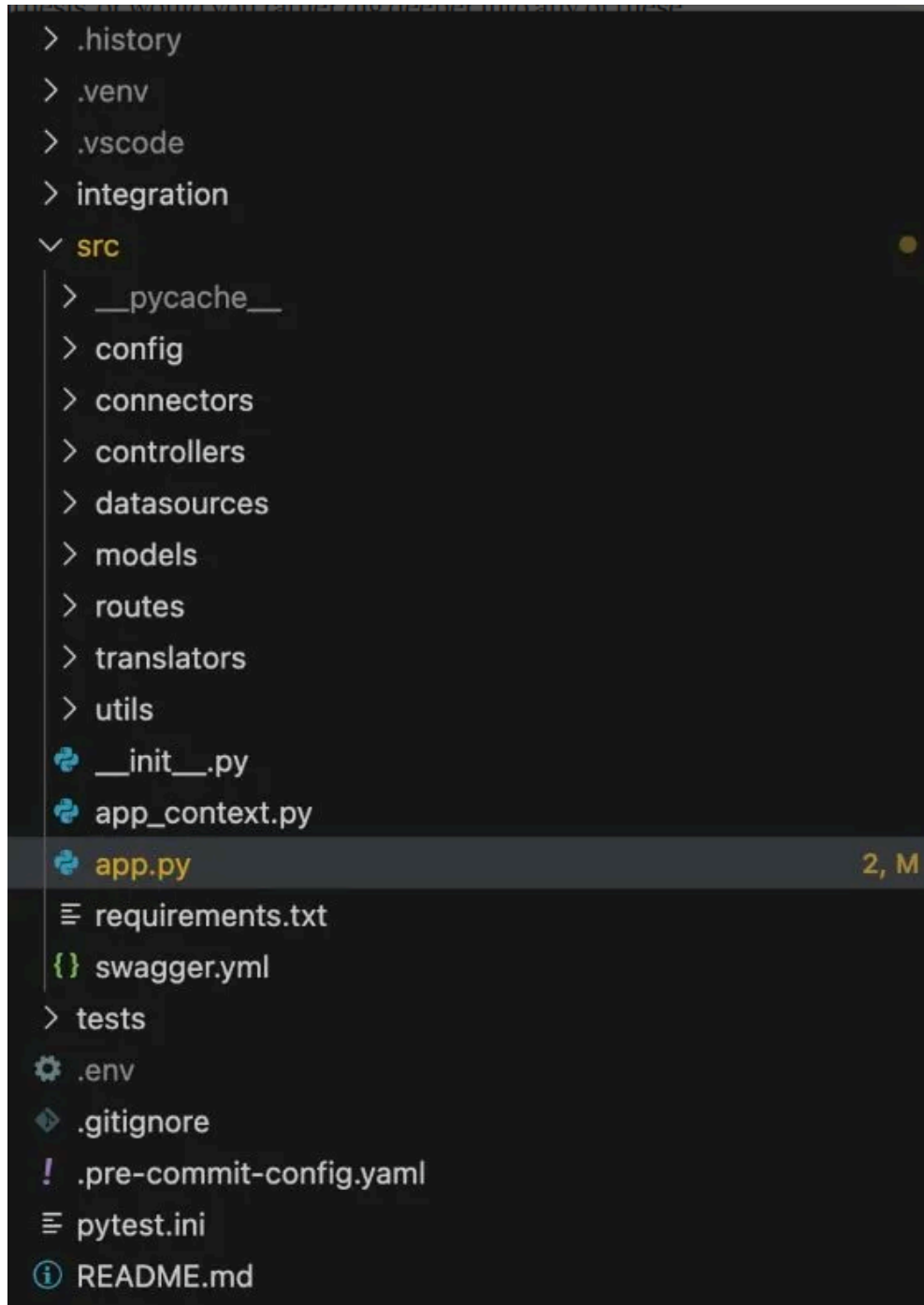


Concept 10: BACAB Architecture Pattern



Imagine a restaurant.

A customer walks in and says "I want a steak, medium rare, with fries." What happens?

The **waiter** takes the order. He doesn't cook. He doesn't know how to cook. He writes down what the customer wants, walks it to the kitchen, and later brings back the plate. That's the **route**.

The **chef** receives the order and makes decisions. "Medium rare means 4 minutes per side. Customer is allergic to peanuts so I'll use olive oil for the fries. We're out of regular fries so I'll substitute sweet potato." The chef knows the rules, makes judgment calls, and coordinates. That's the **controller**.

The **fridge/pantry** is where ingredients are stored and retrieved. The chef says "give me a steak" and the fridge gives a steak. The chef says "store this leftover sauce" and the fridge stores it. No decisions, no cooking — just storage and retrieval. That's the **datasource**.

The **phone on the wall** is for calling outside. The chef needs to call the meat supplier to check if a special cut is available. He doesn't walk to the supplier — he picks up the phone. That's the **connector**.

The **plating station** takes cooked food and arranges it on the plate in a way that looks good for the customer. The kitchen works with pots and pans, but the customer expects a clean plate. The plating station translates between "kitchen format" and "customer format." That's the **translator**.

The **recipe book** describes every ingredient — what a steak is, what fries are, their properties (weight, cooking temperature, origin). That's the **model**.

The **house rules** posted on the wall — opening hours, kitchen temperature, supplier phone numbers — that's the **config**.

The **shared tools** — knives, cutting boards, timers — used by everyone in the kitchen. That's **utils**.

Now let me make it concrete with your project. A user creates a new RFQ.

Step 1 — `routes/rfq_routes.py` (the waiter)

A POST request arrives at `/api/v1/rfqs`. The route file sees it, reads the JSON body, and checks it looks right (has the required fields). It does NOT decide anything. It just says: "Hey controller, someone wants to create an RFQ. Here's the data they sent." Then it waits for the controller to respond so it can deliver the answer back.

python

```
@router.post("/rfqs")
def create_rfq(body: RFQCreateRequest):
    result = rfq_controller.create(body)
    return result
```

Three lines. That's it. A route should always be boring.

Step 2 — `controllers/rfq_controller.py` (the chef)

The controller receives the clean data and starts thinking. This is the only place where "if/then" business logic exists.

python

```
def create(data):
    # 1. Save the RFQ to the database
    new_rfq = rfq_datasource.insert(data)

    # 2. Business rule: log this in history
    history_datasource.insert(rfq_id=new_rfq.id, action="CREATED")

    # 3. Business rule: if a workflow is assigned, auto-create stages
    if data.workflow_id:
        workflow = workflow_datasource.get_by_id(data.workflow_id)
        for stage in workflow.stages:
            rfq_stage_datasource.insert(rfq_id=new_rfq.id, stage=stage)

    # 4. Notify other services that an RFQ was created
    event_connector.publish("rfq.created", new_rfq)

    # 5. Return the result
    return translator.to_response(new_rfq)
```

Notice what's happening — the controller calls multiple datasources (rfq, history, workflow, rfq_stage), calls a connector (event publishing), and uses a translator to format the output. It's the coordinator. It knows the rules but does no database queries itself and sends no HTTP responses itself.

Step 3 — `datasources/rfq_datasource.py` (the fridge)

The controller said "insert this RFQ." The datasource takes the data, talks to PostgreSQL through SQLAlchemy, and returns what it stored.

python

```
def insert(data):
    rfq = RFQModel(
        client_name=data.client_name,
        project_name=data.project_name,
        status="REGISTERED"
    )
    session.add(rfq)
    session.commit()
    return rfq
```

No decisions. No "if the status is X do Y." It just stores and retrieves. If the controller asks for all RFQs with status "IN_PROGRESS", the datasource builds the SQL query, runs it, and returns the rows.

Step 4 — `models/rfq.py` (the recipe book)

This file describes what an RFQ *is* in the database. Column names, types, constraints.

python

```
class RFQModel(Base):
    __tablename__ = "rfq"

    id = Column(UUID, primary_key=True)
    client_name = Column(String, nullable=False)
    project_name = Column(String, nullable=False)
    status = Column(String, default="REGISTERED")
    created_at = Column(DateTime, default=now)
```

This never does anything. It just describes structure. SQLAlchemy reads this to know how to build SQL queries and how to map database rows to Python objects.

Step 5 — `connectors/event_connector.py` (the phone)

The controller needs to tell other services "an RFQ was created." The connector handles that external call — whether it's publishing to a message bus, calling an HTTP API, or sending to Azure Service Bus.

python

```
def publish(event_type, data):
    message = {"type": event_type, "payload": data}
    service_bus.send(message)
```

Tomorrow if you switch from Azure Service Bus to RabbitMQ, you change this one file. No controller changes, no route changes.

Step 6 — `translators/` (the plating station)

The database returns an RFQModel object with SQLAlchemy internals, timestamps as Python datetime objects, UUIDs as UUID objects. The client expects clean JSON with strings. The translator converts between these two worlds.

python

```
def to_response(rfq_model):
    return {
        "id": str(rfq_model.id),
        "client_name": rfq_model.client_name,
        "status": rfq_model.status,
        "created_at": rfq_model.created_at.isoformat()
    }
```

It also works in reverse — converting an incoming JSON request body into the shape the controller and datasource expect.

Step 7 — `config/` (the house rules)

python

```
DATABASE_URL = os.getenv("DATABASE_URL")
IAM_SERVICE_URL = os.getenv("IAM_SERVICE_URL")
EVENT_BUS_CONNECTION = os.getenv("EVENT_BUS_CONNECTION")
```

Loaded once at startup, used everywhere. Nobody reads `.env` directly — they read config.

Step 8 — `utils/` (the shared tools)

python

```
def generate_rfq_number():
    return f"RFQ-{{datetime.now().year}}-{{uuid4().hex[:6]}.upper()}"

def paginate(query, page, size):
    return query.offset((page - 1) * size).limit(size)
```

Small helpers that don't belong to any specific resource.

Step 9 — `app_context.py` (the restaurant manager)

This is the one thing that's easy to overlook but critical. At startup, it creates one instance of every datasource, one instance of every connector, and wires them

into every controller. Instead of each controller creating its own dependencies, the `app_context` hands them out.

python

```
# Create datasources
rfq_ds = RFQDatasource(db_session)
history_ds = HistoryDatasource(db_session)

# Create connectors
event_conn = EventConnector(config.EVENT_BUS_URL)

# Wire into controllers
rfq_controller = RFQController(
    rfq_datasource=rfq_ds,
    history_datasource=history_ds,
    event_connector=event_conn
)
```

Why? Because if you want to test the controller without a real database, you can hand it a fake datasource. The controller doesn't care — it just calls `.insert()` on whatever it was given.

Step 10 — `app.py` (opening the restaurant)

Creates the FastAPI app, registers all routes, starts the server.

python

```
app = FastAPI()
app.include_router(rfq_routes.router)
app.include_router(workflow_routes.router)
# ... etc
'''
```

****The golden rule to remember:**** data flows down, responses flow up, and no layer ever skips a level.

```
'''
route → controller → datasource → database
route → controller → connector → other service
```

Never `route → datasource` directly. Never `datasource → connector`. The controller is always the middleman who knows the rules.

Why This Matters

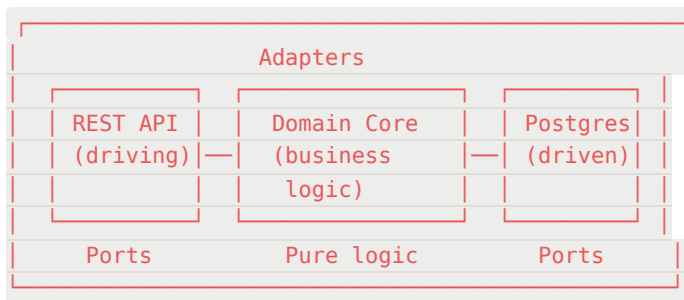
BACAB has its own architectural pattern that is **inspired by hexagonal architecture** but simplified for pragmatic development. Understanding this pattern is essential because every microservice you build must follow it. It's the shared language of your team.

Hexagonal Architecture — Quick Refresher

(See Concept 3 for the full deep-dive)

Hexagonal architecture (Ports & Adapters) organizes code into:

- **Core (Domain):** Business logic, pure — no knowledge of databases or APIs
- **Ports:** Interfaces that define how the outside world interacts with the core
- **Adapters:** Concrete implementations of ports (REST controller, PostgreSQL repository, email sender)



Pure hexagonal requires **explicit port interfaces** — the domain core depends on abstractions, never on concrete implementations.

BACAB's Adaptation — The Pragmatic Version

BACAB takes the hexagonal SPIRIT but drops the formal port interfaces. The result is a **3-layer architecture** that achieves similar separation with less ceremony:

routes/ (API Layer)	controllers/ (Domain Layer)	datasources/ (Infra Layer)
Expose HTTP endpoints	Business logic Orchestration	Database queries
Request/ Response handling	Validation Decisions	External API calls



Mapping to Hexagonal

BACAB Layer	Hexagonal Equivalent	Responsibility
routes/	Driving adapters	HTTP endpoint definitions, request parsing, response formatting
controllers/	Domain / Application core	Business logic, orchestration, validation, decision-making
datasources/	Driven adapters	Database access, external API calls, file system operations

The Full Project Structure (from the image)

```

src/
├─ config/           # Application configuration (env vars, settings, database connection strings)
├─ connectors/       # External system connections (APIs, message brokers, third-party services)
├─ controllers/      # DOMAIN LAYER – business logic and orchestration
├─ datasources/      # INFRASTRUCTURE LAYER – database queries, external data access
├─ models/           # Data structures – entities, DTOs, value objects
├─ routes/           # API LAYER – HTTP endpoint definitions and request handling
├─ translators/      # Data mapping – convert between API shapes and DB shapes
├─ utils/            # Shared helpers – formatting, validation, constants
├─ __init__.py       # Package initialization
├─ app_context.py    # Application context (dependency injection, service registry)
└─ app.py            # Application entry point
  
```

Layer-by-Layer Breakdown

routes/ — The API Exposition Layer

What goes here: HTTP endpoint definitions. Each file typically corresponds to a resource.

python

```

# routes/rfq_routes.py
from flask import Blueprint, request, jsonify
from controllers.rfq_controller import RFQController
  
```



```

rfq_bp = Blueprint('rfq', __name__)
controller = RFQController()

@rfq_bp.route('/rfqs', methods=['GET'])
def list_rfqs():
    filters = request.args # Parse query parameters
    result = controller.list_rfqs(filters) # Delegate to controller
    return jsonify(result), 200 # Format response

@rfq_bp.route('/rfqs', methods=['POST'])
def create_rfq():
    data = request.json # Parse request body
    result = controller.create_rfq(data) # Delegate to controller
    return jsonify(result), 201

```

Rules:

- Routes NEVER contain business logic
- Routes NEVER access the database directly
- Routes only: parse requests → call controller → format responses
- Error handling (HTTP status codes) happens here

controllers/ — The Domain Layer

What goes here: All business logic, validation, orchestration, and decision-making.

python

```

# controllers/rfq_controller.py
from datasources.rfq_datasource import RFQDatasource
from translators.rfq_translator import RFQTranslator

class RFQController:
    def __init__(self):
        self.datasource = RFQDatasource()
        self.translator = RFQTranslator()

    def create_rfq(self, data):
        # Validate business rules
        if not data.get('client_name'):
            raise ValueError("Client name is required")

        # Transform API data to DB format
        entity = self.translator.to_entity(data)

        # Persist

```

```

        created = self.datasource.create(entity)

    # Transform DB result to API format
    return self.translator.to_resource(created)

def advance_stage(self, rfq_id, new_stage):
    rfq = self.datasource.get_by_id(rfq_id)

    # Business logic: validate stage transition
    if not self._is_valid_transition(rfq.current_stage, new_stage):
        raise ValueError(f"Cannot move from {rfq.current_stage} to {new_stage}")

    # Business logic: side effects
    rfq.current_stage = new_stage
    rfq.updated_at = datetime.now()

    self.datasource.update(rfq)
    return self.translator.to_resource(rfq)

```

Rules:

- Controllers contain ALL business logic and decisions
- Controllers call datasources for data access
- Controllers use translators to convert between API and DB formats
- Controllers NEVER know about HTTP (no request/response objects)

datasources/ — The Infrastructure Layer

What goes here: Database queries, external API calls, file operations — anything that touches external systems.

python

```

# datasources/rfq_datasource.py
from models.rfq import RFQ
from config.database import db

class RFQDatasource:
    def get_by_id(self, rfq_id):
        return db.session.query(RFQ).filter(RFQ.id == rfq_id).first()

    def create(self, rfq_entity):
        db.session.add(rfq_entity)
        db.session.commit()
        return rfq_entity

    def list_active(self, filters):
        query = db.session.query(RFQ).filter(RFQ.status == 'active')

```

```

    if filters.get('client'):
        query = query.filter(RFQ.client_name.ilike(f"%{filters['client']}%"))
    return query.all()

```

Rules:

- Datasources ONLY handle data access — no business logic
- Datasources return raw entities/data
- If you need to call an external API, it goes here (or in connectors/)
- Database-specific syntax lives here (SQL, MongoDB queries)

Supporting Layers

models/ — Data structures

python

```

# models/rfq.py
class RFQ(db.Model):
    __tablename__ = 'rfq'
    id = db.Column(db.Integer, primary_key=True)
    reference = db.Column(db.String(50), nullable=False)
    client_name = db.Column(db.String(100), nullable=False)
    current_stage = db.Column(db.String(50))
    status = db.Column(db.String(20), default='active')

```

translators/ — The API Resource ↔ Database Entity bridge

python

```

# translators/rfq_translator.py
class RFQTranslator:
    def to_resource(self, entity):
        """Database entity → API resource (what the client sees)"""
        return {
            "id": entity.id,
            "reference": entity.reference,
            "clientName": entity.client_name, # camelCase for API
            "currentStage": entity.current_stage,
            "status": entity.status
        }

    def to_entity(self, data):
        """API resource → Database entity (what we store)"""
        return RFQ(
            reference=data['reference'],
            client_name=data['clientName'], # snake_case for DB
        )

```

This is where Concept 1 (API Resource vs Database Entity) lives in code. The translator is the physical implementation of that separation.

connectors/ — External system connections

python

```
# connectors/pv_elite_connector.py
class PVEliteConnector:
    def get_vessel_calculations(self, parameters):
        """Call PV ELITE API for pressure vessel calculations"""
        response = requests.post(PV_ELITE_URL, json=parameters)
        return response.json()
```

config/ — Application settings

python

```
# config/database.py
DATABASE_URL = os.getenv('DATABASE_URL', 'postgresql://...')
```

app_context.py — Dependency injection / service registry

python

```
# Manages the lifecycle and wiring of all components
# Which controller uses which datasource, etc.
```

The Flow of a Request



Key Difference from Pure Hexagonal

In pure hexagonal, the controller would depend on a **port interface** (e.g., `RFQRepository` interface), and the datasource would implement that interface. This allows swapping implementations without changing the controller.

In BACAB's approach, the controller directly instantiates the datasource. The **separation is by convention and folder structure**, not by formal interfaces. This is more pragmatic and faster to develop, while still maintaining clean layer boundaries.

Trade-off: You lose the ability to swap datasources without modifying controllers, but you gain simplicity and speed. For most projects (including yours), this trade-off is acceptable.

Key Vocabulary

Term	Definition
Route	An HTTP endpoint definition that maps a URL + method to a controller function
Controller	The domain logic layer that handles business rules and orchestration
Datasource	The infrastructure layer that handles database access and external system calls
Translator	A component that maps between API resource format and database entity format
Connector	A component that manages connections to external systems (APIs, services)
App Context	The central registry that wires together all components and manages dependencies
Blueprint	In Flask, a way to organize routes into modular, reusable groups